

BeyondAllRoyal

Ein mobile-first 2D-RTS in Unity

Inspiziert von *Beyond All Reason* und *Clash Royale*

Idee

Was ist BeyondAllRoyal?

- Genre: 2D Echtzeit-Strategie, mobile-first
- Inspiration: Beyond All Reason + Clash Royale
- Setting: futuristisch
- Modus (MVP): 1 vs 1 gegen eine NPC-KI
- Sieg: feindliches HQ zerstören

Kein Micro nötig

- Einheiten kämpfen vollständig autonom
- Kein Klicken auf jede einzelne Einheit
- Spieler entscheidet was gebaut wird, nicht wie gekämpft wird

Zielpriorität der Einheiten

1. Feinde auf eigener Kartenseite
2. Gebäude in Reichweite
3. Einheiten in Reichweite
4. Gebäude ausser Reichweite
5. Einheiten ausser Reichweite

Bewegung nur nach vorne, nie zurück, um Feinde zu jagen. Verhindert dass die ganze Armee umdreht.

Ressource: Metal

- Globaler Pool
- Sofort fällig bei Bau- oder Produktionsstart
- Gemeinsames Bottleneck für alle Gebäude

Ressource: Energy

- Pro Gebäude ein eigener Puffer
- Füllt sich passiv über Zeit
- Fertig, sobald genug Energy da ist
- **Kein fixer Timer**
- Tesla Tower beschleunigt Nachbargebäude

Konter-System

5 Einheiten, 2 Türme

Jede Einheit kontert genau 2, wird von genau 2 gekontert

Einheit	Kontert	Gekontert von
Soldier	Gunner, Explosive	Hovercraft, Tank
Heavy Gunner	Explosive, Hovercraft	Tank, Soldier
Explosive Spec.	Hovercraft, Tank	Soldier, Gunner
Hovercraft	Tank, Soldier	Gunner, Explosive
Heavy Tank	Soldier, Gunner	Explosive, Hovercraft

Keine All-Rounder

Demo

Technik

Prinzip: Daten statt Code

- Alle Werte leben in `ScriptableObject`s
- `UnitData`, `BuildingData`, `CounterChartData`, `MapLayoutData`
- Code liest nur, enthält keine Zahlen
- Balancing ändern, ohne Code anzufassen

Muster: Vererbung für Gebäude

- Basisklasse `Building`: Energie, Bau, Schaden, Grid-Position
- Darauf aufbauend: Production, Tower, Tesla, Factory, HQ
- `HQ` = `MetalFactory` + `TeslaTower` + Selbstverteidigung
- Wiederverwendung statt Copy-Paste

Muster: Registry statt Suche

- Naiv: jeden Frame die ganze Szene durchsuchen
- Stattdessen: `UnitRegistry`, `BuildingRegistry`
- Einheiten melden sich selbst an/ab

Prinzip: gleiche Werte, unterschiedliche Taktik

- Einheiten-/Gebäudewerte bleiben für Spieler und KI immer gleich
- Easy/Medium: 3 zufällige Gebäudetypen, festes Tempo
- Hard: alle 5 Typen, baut reaktiv gegen die häufigste gegnerische Einheit

Test

Wie getestet wird

- `TestScene`: vorplatziertes Start-Loadout
- Kein Aufbau von Null bei jedem Testlauf
- Manuelles Playtesting und Balancing
- Pre-Commit-Hook scannt auf Secrets/Keys

Reflexion

Was gut lief

- Daten statt Code für schnelles Tweaking
- Autonome Einheiten
- Testen mit Testscene

Herausforderungen

- Metal und Energy als zwei Bottlenecks: viel Iteration und Tuning
- UI kostet viel Zeit
- Konter-System schwierig intuitiv darzustellen

Nächste Schritte

- Echte Sprites und Sounds statt Platzhalter
- Breiteres Playtesting für KI-Schwierigkeit und Balancing
- UI verschönern
- Multiplayer als nächster grosser Meilenstein

Fragen?

Danke für die Aufmerksamkeit